



中国矿业大学
CHINA UNIVERSITY OF MINING AND TECHNOLOGY

《大数据可视化》综合设计报告二

题目：传统机器学习基础方法实践- 鲍鱼年龄预测

学 院 计算机科学与技术学院

班 级 2023 级大数据一班

姓 名 王志豪

学 号 08232337

中国矿业大学

2026 年 05 月

课程设计报告格式要求：

1. 报告中应包含一、二级标题；
2. 正文宋体小 4 号，1.5 倍行距；
3. 报告正文不少于 1000 字；
4. 上交报告电子版：<https://v.wjx.cn/vm/mv3f28U.aspx>，截止 4 月 26 日。

课程设计报告评分要点：

1. 报告中可视化技术的应用情况（20%）
2. 报告中可视化应用功能有效及完整性（40%）
3. 报告中的个人总结及体会情况（20%）
4. 报告格式规范程度（20%）

评语：

评分：

日期：

目录

1. 背景及功能概要	3
1.1 背景.....	3
1.2 功能概要.....	3
2. 代码设计.....	4
3. 实现及验证.....	6
4. 总结及体会.....	9

1. 背景及功能概要

1.1 背景

随着人工智能和机器学习技术的快速发展，利用计算机对实际问题进行预测与分析已成为数据科学的重要研究方向。回归分析作为监督学习中的经典方法，被广泛应用于房价预测、销售预测、生物信息分析等领域。其中，线性回归模型因其原理简单、易于实现且具有较好的可解释性，成为机器学习入门和实践中最基础、最重要的模型之一。线性回归通过建立自变量与因变量之间的线性关系，利用历史数据学习模型参数，从而实现对未知数据的预测。

本项目以鲍鱼年龄预测为研究对象，基于 PaddlePaddle 深度学习框架构建线性回归模型。鲍鱼年龄与其长度、直径、高度、重量等物理特征之间存在一定的关联关系，因此可以利用这些特征作为模型输入，对鲍鱼年龄进行预测。在模型训练过程中，通过假设函数建立输入特征与预测结果之间的映射关系，并采用均方误差（MSE）作为损失函数衡量预测值与真实值之间的差异，再利用优化算法不断调整模型参数，使损失函数逐步收敛，从而获得较好的预测效果。通过本项目的实践，可以深入理解线性回归模型的基本原理、模型训练流程以及 PaddlePaddle 框架在机器学习任务中的应用方法。

1.2 功能概要

- (1). 数据预处理：读取文本数据；把性别做映射编码；数值特征转 float；拼接特征与标签；按列做 min-max 归一化；划分训练/验证/测试集。
- (2). 网络配置：用一层全连接回归模型（输入 8 维，输出 1 维）；定义平方误差损失；使用 SGD 优化器。
- (3). 网络训练：按 batch 迭代训练，执行前向与反向更新参数；记录每次迭代 loss；每个 epoch 结束做一次验证。
- (4). 网络评估：在验证集上用 [test_program](#) 计算平均 loss；更新并保存验证集 loss 最低的模型参数。
- (5). 网络预测：加载最佳模型参数；在测试集上推理得到预测值；计算平均绝对误差并绘制散点图。

2. 代码设计

由于本项目中的 Paddle 包版本很老（1.x 的古老版本），因此所有的流程都是按照老版本的方法进行编写的。

(1). 数据预处理

```
file_path = "data/data361/AbaloneAgePrediction.txt"

raw_data = []
with open(file_path, "r") as f:
    for line in f:
        line = line.strip()
        if not line:
            continue
        raw_data.append(line.split(","))
sex_map = {"M": 0, "F": 1, "I": 2}
features, labels = [], []
for row in raw_data:
    sex = sex_map[row[0]]
    numeric_features = list(map(float, row[1:-1]))
    x = [sex] + numeric_features
    y = float(row[-1])
    features.append(x)
    labels.append(y)
X = np.array(features, dtype=np.float32)
y = np.array(labels, dtype=np.float32)
x_min = X.min(axis=0)
x_max = X.max(axis=0)
X_norm = (X - x_min) / (x_max - x_min + 1e-8)
X_temp, X_test, y_temp, y_test = train_test_split(X_norm, y, test_size=10,
random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.2,
random_state=42)
def train_reader():
    for i in range(len(X_train)):
        yield X_train[i], y_train[i]
train_loader = paddle.batch(train_reader, batch_size=32)
```

先读入文本并按行切分，得到原始字符串特征；性别做离散编码后与数值特征拼接，构成 8 维输入；标签取最后一列。随后对每一列做 min-max 归一化以消除量纲差异。最后按固定随机种子划分训练/验证/测试集，并用旧版 reader 生成批数据供训练阶段使用。

(2). 网络配置

```
x = fluid.layers.data(name='x', shape=[8], dtype='float32')
y = fluid.layers.data(name='y', shape=[1], dtype='float32')
predict = fluid.layers.fc(input=x, size=1, act=None)
loss = fluid.layers.square_error_cost(input=predict, label=y)
avg_loss = fluid.layers.mean(loss)
optimizer = fluid.optimizer.SGD(learning_rate=0.01)
optimizer.minimize(avg_loss)
test_program = fluid.default_main_program().clone(for_test=True)
place = fluid.CPUPlace()
exe = fluid.Executor(place)
exe.run(fluid.default_startup_program())
```

输入为 8 维特征，输出为 1 维连续值，使用单层全连接实现线性回归；损失函数为平方误差并取均值；优化器用 SGD，学习率 0.01。通过克隆主图得到 test_program，确保评估阶段不更新参数。执行器与启动程序只初始化一次，参数在训练过程中持续更新。

(3). 网络训练

```
for epoch_id in range(EPOCH_NUM):
    for batch_id, data in enumerate(train_loader()):
        x_data = np.array([item[0] for item in data]).astype('float32')
        y_data = np.array([item[1] for item in
data]).astype('float32').reshape(-1, 1)
        avg_loss_value = exe.run(
            program=fluid.default_main_program(),
            feed={'x': x_data, 'y': y_data},
            fetch_list=[avg_loss]
        )
```

训练阶段按 epoch 迭代，每个 batch 组装为二维输入与列向量标签。exe.run 在主训练图上执行前向与反向传播并更新参数，同时取出平均损失用于监控。训练损失被记录到列表用于后续可视化。

(4). 网络评估

```
test_costs = []
for _, data in enumerate(val_loader()):
    x_data = np.array([item[0] for item in data]).astype('float32')
    y_data = np.array([item[1] for item in data]).astype('float32').reshape(-1,
1)
    test_cost = exe.run(
        program=test_program,
        feed={'x': x_data, 'y': y_data},
```

```

        fetch_list=[avg_loss]
    )
    test_costs.append(test_cost[0][0])
test_cost = float(sum(test_costs) / len(test_costs))
if test_cost < best_cost:
    best_cost = test_cost
    fluid.io.save_params(executor=exe, dirname=save_dir,
main_program=fluid.default_main_program())

```

评估阶段使用 `test_program`，仅做前向计算不会更新权重。把每个 `batch` 的损失累加求均值，得到该 `epoch` 的验证损失。若当前损失优于历史最佳，就保存当前参数作为“最佳模型”。

(5). 网络预测

```

fluid.io.load_params(executor=exe, dirname="./model")
for batch_id, data in enumerate(test_loader()):
    x_data = np.array([item[0] for item in data]).astype('float32')
    y_data = np.array([item[1] for item in data]).astype('float32').reshape(-1,
1)
    result = exe.run(
        program=test_program,
        feed={'x': x_data, 'y': y_data},
        fetch_list=[predict]
    )
infer_results = np.array(infer_results)
ground_truths = np.array(ground_truths)
avg_error = np.mean(np.abs(infer_results - ground_truths))

```

先加载验证集上表现最好的参数，再在测试集上做推理。推理阶段只取 `predict` 输出，随后把预测值与真实值对齐计算平均绝对误差，最后绘制散点图用于直观观察预测效果。

3.实现及验证

(1). 数据预览

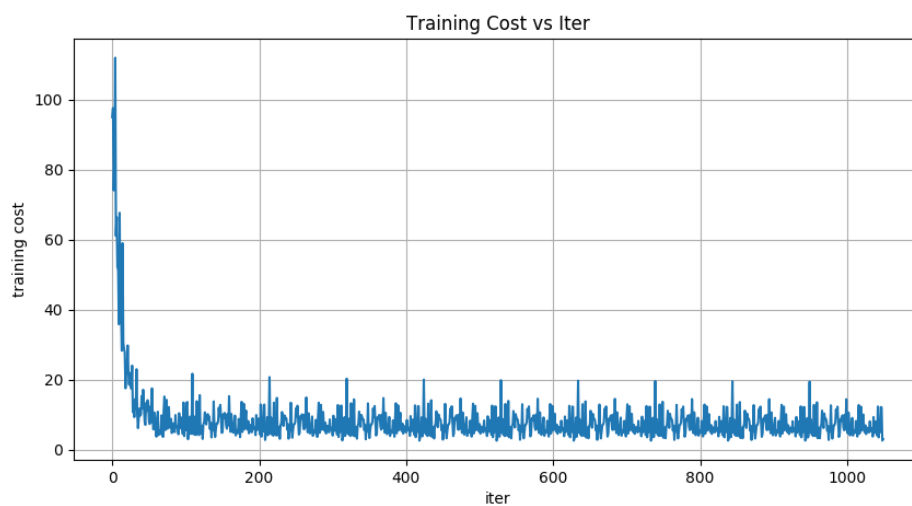
样本数量：4177

第一条数据：['M', '0.455', '0.365', '0.095', '0.514', '0.2245', '0.101', '0.15', '15']


```
X shape: (4177, 8)
y shape: (4177,)
[[0.          0.5135135  0.52100843 0.08407079 0.18133524 0.15030262
  0.1323239  0.14798208]
 [0.          0.37162158 0.35294116 0.07964602 0.07915708 0.06624075
  0.06319948 0.06826109]
 [0.5         0.6148648  0.6134454  0.11946903 0.239065   0.17182247
  0.18564847 0.20777279]
 [0.          0.49324325 0.52100843 0.11061947 0.18204357 0.14425017
  0.14944042 0.15296464]
 [1.          0.3445946  0.33613446 0.07079646 0.07189658 0.0595158
  0.05134958 0.0533134 ]]
训练集大小: (3333, 8)
测试集大小: (10, 8)
验证集大小: (834, 8)
```

(2). 训练过程与结果

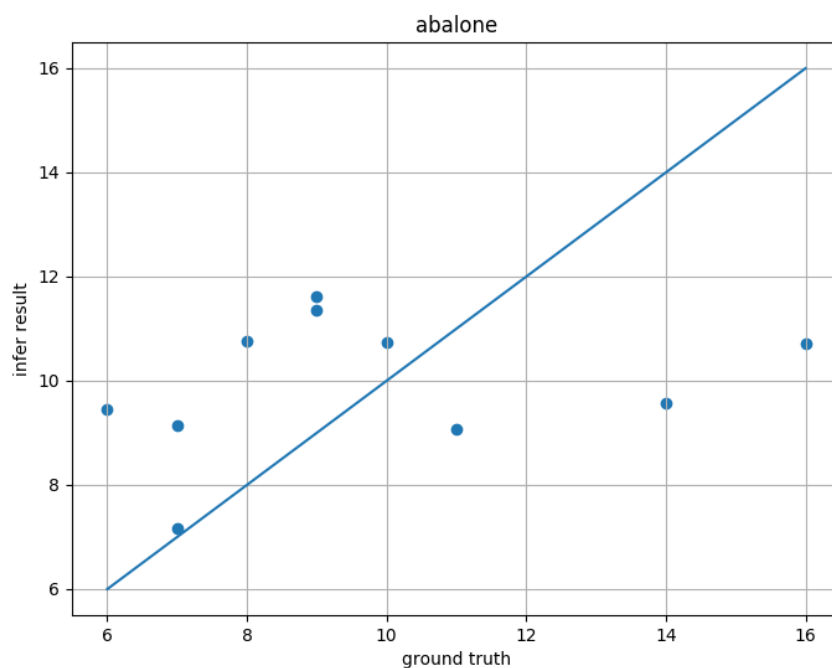
```
epoch: 2, iter: 100, loss is: 5.011521339416504
Test Epoch:2, Cost:6.19541
epoch: 3, iter: 0, loss is: 4.4287824630737305
epoch: 3, iter: 50, loss is: 6.566449165344238
epoch: 3, iter: 100, loss is: 4.958245277404785
Test Epoch:3, Cost:6.14257
epoch: 4, iter: 0, loss is: 4.353004455566406
epoch: 4, iter: 50, loss is: 6.538318634033203
epoch: 4, iter: 100, loss is: 4.93703556060791
Test Epoch:4, Cost:6.10645
epoch: 5, iter: 0, loss is: 4.308640480041504
epoch: 5, iter: 50, loss is: 6.515207290649414
epoch: 5, iter: 100, loss is: 4.929480075836182
Test Epoch:5, Cost:6.07586
epoch: 6, iter: 0, loss is: 4.279354095458984
epoch: 6, iter: 50, loss is: 6.491999626159668
epoch: 6, iter: 100, loss is: 4.927972316741943
Test Epoch:6, Cost:6.04728
epoch: 7, iter: 0, loss is: 4.257908821105957
epoch: 7, iter: 50, loss is: 6.46775484085083
epoch: 7, iter: 100, loss is: 4.929250717163086
Test Epoch:7, Cost:6.01979
epoch: 8, iter: 0, loss is: 4.240894317626953
epoch: 8, iter: 50, loss is: 6.442730903625488
epoch: 8, iter: 100, loss is: 4.931849479675293
Test Epoch:8, Cost:5.99326
epoch: 9, iter: 0, loss is: 4.226583480834961
epoch: 9, iter: 50, loss is: 6.4173808097839355
epoch: 9, iter: 100, loss is: 4.935068130493164
Test Epoch:9, Cost:5.96773
最佳模型保存成功!
```



(3). 测试结果

No. 0: infer result is 11.37, ground truth is 9.00
 No. 1: infer result is 10.76, ground truth is 8.00
 No. 2: infer result is 10.70, ground truth is 16.00
 No. 3: infer result is 11.62, ground truth is 9.00
 No. 4: infer result is 9.57, ground truth is 14.00
 No. 5: infer result is 9.06, ground truth is 11.00
 No. 6: infer result is 9.13, ground truth is 7.00
 No. 7: infer result is 9.45, ground truth is 6.00
 No. 8: infer result is 7.17, ground truth is 7.00
 No. 9: infer result is 10.75, ground truth is 10.00

平均误差为: [2.5905994415283202]



4.总结及体会

本实验基于 PaddlePaddle 深度学习框架完成了鲍鱼年龄预测任务，实现了从数据读取、数据预处理、训练集与测试集划分、神经网络模型构建到模型训练、模型评估和结果可视化的完整流程。实验中采用全连接神经网络对鲍鱼的物理特征进行学习，并利用均方误差作为损失函数对模型进行优化。通过观察训练损失曲线以及测试集预测结果，可以发现模型能够较好地学习数据中的规律，实现对鲍鱼年龄的有效预测。同时，通过绘制真实值与预测值的散点图，更直观地验证了模型的预测性能和泛化能力。

通过本次实验，我进一步掌握了 PaddlePaddle 框架的基本使用方法，加深了对神经网络训练流程和回归预测任务的理解。在实验过程中，不仅学习了数据归一化、模型参数保存与加载、训练过程可视化等常用技术，还学会了根据报错信息定位和解决程序问题，提高了调试和分析能力。实践表明，数据质量、模型结构以及参数设置都会对最终预测效果产生影响，今后还可以通过调整网络层数、优化器参数以及特征工程等方式进一步提升模型性能。